Retrieval-Augmented Generation (RAG) using Vector Database and LLM

GENAI EX-7.ipynb

AIM:

Develop a Python program to integrate a Vector Database with a Large Language Model (LLM) to build a Retrieval-Augmented Generation (RAG) system that answers questions based on external PDF documents such as LinkedIn profiles or research papers.

CODE:

```
# Install once
!pip install -q pymupdf sentence-transformers faiss-cpu transformers

import fitz, faiss, numpy as np
from google.colab import files
from sentence_transformers import SentenceTransformer
from transformers import pipeline

# Upload PDF
up = files.upload()
pdf = list(up.keys())[0]

# Extract text
text = ""
doc = fitz.open(pdf)
for page in doc:
   text += page.get_text()

# Split into chunks
chunks = [text[i:i+500] for i in range(0, len(text), 500)]

# Embeddings
emb_model = SentenceTransformer("all-MiniLM-L6-v2")
emb = emb_model.encode(chunks)
```

```python
# FAISS index
index = faiss.IndexFlatL2(emb.shape[1])
index.add(np.array(emb))

# QA model
qa = pipeline("question-answering")

# RAG function
def rag(query):
    q_emb = emb_model.encode([query])
    D, I = index.search(np.array(q_emb), k=2)
    context = " ".join([chunks[i] for i in I[0]])
    ans = qa(question=query, context=context)["answer"]
    return context, ans

# Ask repeatedly
while True:
    query = input("Enter your question (or type exit): ")
    if query.lower() == "exit":
        break
    context, answer = rag(query)
    print("\nRetrieved Context:\n", context)
    print("\nAnswer:\n", answer)
    print("\n" + "-"*60)
```

OUTPUT:

```
Enter your question (or type exit): What is transformer?

Retrieved Context:
 at variety of Transformer variants (a.k.a. X-formers)
have been proposed, however, a systematic and comprehensive literature review on these Transformer variants
is still missing. In this survey, we provide a comprehensive review of various X-formers. We first briefly
introduce the vanilla Transformer and then propose a new taxonomy of X-formers. Next, we introduce the
various X-formers from three perspectives: architectural modification, pre-training, and applications. Finally,
we outline some  t perspectives, such as efficiency, gen-
eralization, and applications. The improvements include incorporating structural prior, designing
lightweight architecture, pre-training, and so on.
Although X-formers have proven their power for various tasks, challenges still exist. Besides
the current concerns (e.g. efficiency and generalization), the further improvements of Transformer
may lie in the following directions:
(1) Theoretical Analysis. The architecture of Transformer has been demonstrated

Answer:
 vanilla Transformer

------------------------------------------------------------
Enter your question (or type exit): What are the applications of Transformer?

Retrieved Context:
 t perspectives, such as efficiency, gen-
eralization, and applications. The improvements include incorporating structural prior, designing
lightweight architecture, pre-training, and so on.
Although X-formers have proven their power for various tasks, challenges still exist. Besides
the current concerns (e.g. efficiency and generalization), the further improvements of Transformer
may lie in the following directions:
(1) Theoretical Analysis. The architecture of Transformer has been demonstrated  [61, 94] and video
processing [3, 115]. Han et al. [47] and Khan et al. [64] provide reviews on existing work of visual
Transformers. We encourage readers to refer to these surveys for further understand the current
research progress on Transformers in CV.
(3) Audio Applications. Transformer can also be extended for audio-related applications, e.g.,
speech recognition [15, 31, 41, 97], speech synthesis [57, 76, 169], speech enhancement [65, 162]
and music generation [56].
(4) Multimodal Applicat

Answer:
 audio-related applications

------------------------------------------------------------
Enter your question (or type exit): exit
```